

L&T CIA - 3

Advanced JavaScript Backend Frameworks
(Node.js & Express JS)

P07 - Restaurant Table Reservation & Food Ordering System

Team Details

Course Name: Advanced JavaScript Backend Frameworks
(Node.js & Express JS)

Section: 5BTCS AIML A

Semester: 5

Team No. : 09

S.No	Student Name	Roll No.	Department	Section
1	Abel Alexander	2462004	ADSE	5BTCS AIML A
2	Abishek K.S	2462013	ADSE	5BTCS AIML A
3	Abhishek Nath	2462007	ADSE	5BTCS AIML A
4	Aanon Thomas Lino	2462001	ADSE	5BTCS AIML A

GitHub Repository Link:

<https://github.com/Abishek-gitit/restaurant-reservation-system>

Contents

1	Abstract and Introduction	1	9	Food Ordering and Price Preservation	9
	Abstract	1		Supported Order Types	9
	Background and Need	1		Order Validation Sequence	9
	Project Approach	1		Authoritative Price Calculation	9
	Objectives and Scope	1		Embedded Purchase Snapshots	9
2	Requirements and Technology Stack	2		Order Access and Follow-Up	9
	Functional Requirements	2	10	Kitchen Queue and Order Workflow	10
	Technology Selection	2		Kitchen Display Queue	10
	Quality Requirements	2		Permitted State Transitions	10
	Execution Environment	2		Role-Specific Actions	10
3	System Architecture and API Design	3		Audit History and Coordination	10
	Layered Request Processing	3	11	Billing and Order Summaries	11
	Responsibilities of Each Layer	3		Centralized Billing Rules	11
	REST Resource Organization	3		Illustrative Bill	11
	Consistent Error Responses	3		Bill and Summary Endpoints	11
4	Database Design and Data Integrity	4		Privacy and Interpretation	11
	Main Collections	4	12	Customer History and Verified Feedback	12
	References and Embedded Records	4		Paginated Histories	12
	Indexes and Uniqueness	4		Feedback Eligibility	12
	Integrity Beyond the Schema	4		Review Workflow	12
5	Authentication and Access Control	5		Value to Customers and Managers	12
	Registration and Login	5	13	Manager Analytics and Frontend Dash- board	13
	Role-Based Permissions	5		Branch-Scoped Reports	13
	Ownership and Branch Scoping	5		Aggregation Approach	13
	Data Exposure and Validation	5		Role-Oriented Browser Interface	13
	Security Boundary	5		Interpreting Operational Information	13
6	Branch, Menu and Table Management	6	14	Setup, Demonstration and Testing	14
	Branch Management	6		Documented Setup Procedure	14
	Menu Administration	6		Suggested Demonstration Sequence	14
	Table Inventory	6		Reported Verification Coverage	14
	Operational Example	6		Verification Limit	14
	Deactivation Versus Deletion	6	15	Outcomes, Limitations and Conclusion	15
7	Table Reservation Engine	7		Documented Outcomes	15
	Reservation Creation	7		Limitations and Documentation Gaps	15
	Overlap Detection	7		Possible Future Enhancements	15
	End-Time Exclusivity	7		Conclusion	15
	Lifecycle and Ownership	7			
8	Cancellation and Rescheduling	8			
	Customer Cancellation Policy	8			
	Illustrative Boundary Cases	8			
	Rescheduling Procedure	8			
	Preserving the Original Booking	8			

1 Abstract and Introduction

Abstract

The Restaurant Table Reservation & Food Ordering System is a multi-branch restaurant application that brings table bookings, food orders, kitchen coordination, billing, and customer feedback into one workflow. Developed for the fifth-semester CIA-3 assessment at Christ University, the project uses Node.js and Express.js for the backend, MongoDB with Mongoose for data storage, and HTML, CSS, and vanilla JavaScript for its browser interface. Its main purpose is to reduce inconsistencies between restaurant seating, customer reservations, and food preparation.

The application supports four user roles: customer, kitchen staff, manager, and administrator. Customers can reserve tables, place dine-in or takeaway orders, review their history, and submit feedback after an order is completed. Kitchen staff receive an oldest-first preparation queue. Managers oversee their assigned branches and access operational reports, while administrators retain system-wide access. Authentication, ownership checks, and branch-level authorization control access to these functions.

Background and Need

Manual restaurant operations often separate table bookings from ordering and billing. A booking recorded by one staff member may not be visible to another, increasing the risk of overlapping reservations. Similar problems arise when menu availability changes but customers or servers continue to use an outdated list. In a restaurant with multiple branches, these inconsistencies become harder to coordinate.

A shared backend helps each operation use the same branch, table, menu, reservation, and order records. The project combines time-slot conflict detection with controlled order status changes so that accepting a booking and preparing a meal follow explicit rules rather than informal communication. Historical item prices are also retained so that a later menu update does not alter the original purchase record.

Project Approach

The system follows an MVC-oriented REST architecture with middleware for input validation, token verification, and authorization. Business rules cover reservation overlap, a two-hour customer cancellation window, item availability, server-side billing, and eligibility for verified reviews. MongoDB aggregation reports provide summaries of sales, popular dishes, demand periods, and customer ratings.

Objectives and Scope

The main objectives are to prevent overlapping reservations, validate branch-specific orders, preserve original purchase prices, and coordinate staff actions through controlled status changes. The system also aims to provide customers with useful histories and verified feedback, and managers with reports for their assigned branches.

Development is organized into four phases: authentication and inventory foundations; reservations and ordering; kitchen operations, billing, and cancellation; and finally histories, feedback, branch controls, and analytics. The scope covers this integrated restaurant workflow, not external payment settlement or courier tracking. This report presents the architecture, business rules, and operational workflows at a moderate level of detail. Reported implementation and testing outcomes are distinguished from independently verified results.

2 Requirements and Technology Stack

Functional Requirements

Customers require account registration and login, branch and menu browsing, table booking, food ordering, access to personal history, and feedback submission. Kitchen staff require a branch-filtered queue with item quantities and permitted preparation updates. Managers require tools for branch operations and reports, while administrators require global oversight. All of these functions depend on consistent links between users, branches, tables, and transactions.

The backend must reject invalid quantities, conflicting bookings, unavailable items, unauthorized changes, and reviews for unfinished orders. Successful operations should return the relevant result, while failures should use a predictable error structure that the dashboard can interpret.

Technology Selection

Technology	Role in the Project
Node.js and Express.js	JavaScript runtime and HTTP framework for REST routes, middleware, and backend request handling.
MongoDB and Mongoose	Document storage, schemas, references, validation, and database indexes.
JWT and bcryptjs	Signed-token authentication and password hashing with 10 salt rounds.
Joi	Request-schema validation before business operations proceed.
HTML, CSS, vanilla JS	Browser-based single-page dashboard served by Express from the <code>public/</code> directory.
dotenv, morgan, cors	Environment configuration, development request logging, and cross-origin request middleware.

Quality Requirements

Security depends on distinguishing authentication from authorization: a valid login alone should not provide access to every record. Consistency requires that ordering and reservation rules be checked on the server. Maintainability is supported by separating routes, middleware, controllers, services, and models rather than placing every responsibility in one handler.

The design also uses indexed queries and pagination to support growing transaction histories. These are design choices intended to improve efficiency; they are not a substitute for measured load-test results. The report does not claim a specific response time or number of concurrent users.

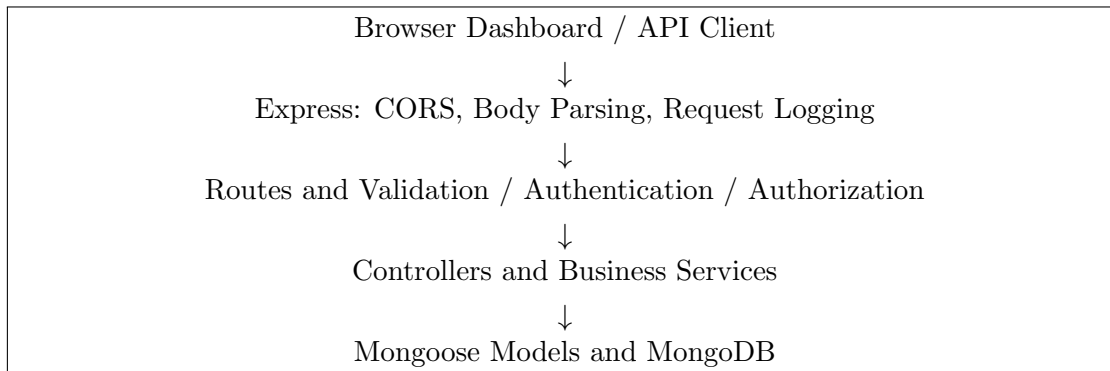
Execution Environment

The technology section identifies Node.js v20+, Express.js v4, and Mongoose v8, while the installation section lists Node.js v18+ and npm v9+ as prerequisites. This is a version inconsistency in the source documentation. For reproducing the described setup, Node.js v20+ follows the stricter stated baseline, but the actual application package configuration should be checked before deployment. MongoDB may be local, hosted through Atlas, or provided by the documented automated fallback.

3 System Architecture and API Design

Layered Request Processing

The application uses an MVC-oriented structure adapted to REST APIs. The browser dashboard acts as the user-facing view, controllers handle incoming actions, and Mongoose models represent stored data. Services contain operations such as reservation checks, billing calculations, and report aggregation. This arrangement keeps presentation concerns separate from business rules and persistence.



Responsibilities of Each Layer

Global middleware handles shared HTTP concerns before requests reach their routes. Route-level middleware then applies the relevant validation schema, verifies the token where required, checks user roles, and enforces branch access. Controllers and services perform the requested business operation using validated inputs and the authenticated identity. Mongoose models provide the database-facing representation and index definitions.

For example, a reservation request must pass access and input checks before the service verifies branch membership, future timing, and overlap. Only an acceptable request should produce a reservation record. A failure at any stage is returned through the application’s common error-handling mechanism.

REST Resource Organization

Routes are grouped by responsibility: `/api/auth` manages identity, `/api/branches`, `/api/menu`, and `/api/tables` expose restaurant resources, and `/api/reservations` and `/api/orders` handle transactions. Dedicated route groups cover customers, kitchen operations, feedback, and manager reports. This makes the API easier to navigate without requiring the client to understand database internals.

Consistent Error Responses

Failure responses use `success`, `message`, `errorCode`, and `details`. HTTP 400 represents invalid input, 401 an authentication failure, 403 denied access, 404 a missing resource, and 409 a conflicting operation. Unexpected errors use HTTP 500 with a safe response and internal logging. Machine-readable codes let the dashboard distinguish, for example, an unavailable menu item from a reservation conflict and display an appropriate message.

4 Database Design and Data Integrity

Main Collections

The database separates independently managed entities while linking them through identifiers. The following table summarizes the documented model rather than reproducing a complete schema.

Collection	Responsibility and Relationships
User	Identity, role, and manager branch assignments through managedBranchIds .
Branch	Restaurant location details and operational state through isActive .
Table and MenuItem	Branch-linked seating inventory and menu catalog, including capacities, prices, and availability.
Reservation	Customer, branch, table, booking times, duration, status, and status history.
Order	Customer and branch links, order type, reservation link where applicable, item snapshots, and workflow history.
Feedback	Order, customer, and branch links with a rating and optional review comment.

References and Embedded Records

References allow tables, menus, customers, and bookings to be queried and maintained independently. Storing every transaction inside a single branch document would create an increasingly large record and make routine updates less manageable. Separate collections instead preserve the branch relationship without collecting its entire operating history in one document.

Order items are embedded because they describe a specific purchase. Each snapshot contains the menu item identifier, name, unit price, quantity, and line total. If a menu price changes later, the order still retains the terms accepted when it was placed. Order and reservation status histories are also embedded so the sequence of actions remains attached to its parent record.

Indexes and Uniqueness

Unique indexes protect user email addresses and branch names. The table index combines branch identity with table number, allowing different branches to each have a Table 1 while rejecting duplicates within a branch. Feedback uses a unique order–customer pair alongside ownership checks to prevent repeated reviews of the same purchase.

Reservation indexes include table and start time for conflict searches, while customer-and-date indexes support history retrieval. Orders use branch, status, and creation time for operational queries. These indexes reflect common access patterns; exact query performance would need examination against the implemented queries and actual data.

Integrity Beyond the Schema

References and indexes do not replace business validation. The backend must still ensure that an order’s items belong to its selected branch and that a reservation’s table belongs to the chosen location. Integrity therefore combines database constraints, model validation, and service-level checks.

5 Authentication and Access Control

Registration and Login

Public registration creates customer accounts with the customer role enforced by the application. A registration request should not gain staff privileges merely by supplying a different role value. Passwords are handled through bcryptjs hashing, and successful login returns a signed JSON Web Token. Protected requests use the token to establish the caller's identity. The `/api/auth/me` endpoint returns the authenticated user's profile.

Role-Based Permissions

Role	Documented Access
Customer	Own reservations, food orders, bills, histories, and eligible feedback; no staff preparation-status updates.
Kitchen staff	Active kitchen queue and progression from placed to preparing and from preparing to ready; no financial bill access.
Manager	Branch operations, permitted order and reservation actions, and reports within assigned branch scope.
Administrator	Global management access and oversight across restaurant branches.

Ownership and Branch Scoping

Role checks alone are insufficient when two users share the same role. A customer must not retrieve another customer's order simply by changing an identifier in a request. Ownership checks connect the requested resource to the authenticated customer before permitting access or modification. The same principle applies to reservations and feedback.

Managers are constrained by `User.managedBranchIds`. A manager assigned to one branch should not inspect or modify another branch's restricted information. Out-of-scope requests return a forbidden or branch-access-denied response. Administrators retain global system access. Together, role, ownership, and branch checks establish more precise permissions than a single authenticated/unauthenticated distinction.

Data Exposure and Validation

The kitchen queue presents operational details such as table numbers, order types, items, and quantities while removing customer names, email addresses, and customer identifiers. Financial endpoints are also restricted so that preparation staff do not receive billing information unnecessarily. These controls align the returned data with the user's task.

Joi and Mongoose validation provide additional checks on request shape and stored values. Examples include invalid identifiers, out-of-range ratings, malformed date filters, and incorrect quantities. Missing or invalid tokens produce HTTP 401, while authenticated users without the required permission receive HTTP 403.

Security Boundary

The academic setup includes demonstration accounts and a sample token secret. These examples are not appropriate production credentials. Secret replacement, deployment configuration, and broader operational hardening would need attention before using the application with real customer data.

6 Branch, Menu and Table Management

Branch Management

A branch represents a restaurant location with its own seating and menu resources. The system supports creating, listing, retrieving, updating, and deleting branch records, with unique branch names and branch-related access controls. Customers can browse branches publicly, while management operations require the appropriate staff permissions.

The operational flag `isActive` is distinct from deleting a branch. Setting a branch inactive prevents new reservations and new food orders with HTTP 409 and the code `BRANCH_INACTIVE`. Existing historical records remain available for reference and reporting. This is useful when a location temporarily stops accepting business without discarding its previous activity.

Menu Administration

Menu items are linked to a branch and organized into categories such as Starters, Main Course, Desserts, and Beverages. The menu listing supports branch and category filters, allowing customers to view relevant dishes rather than a combined catalog from every location. Authorized staff can update pricing and availability as operations change.

Availability is checked again when an order is placed. A dish shown in a customer's earlier browser session might have become unavailable before checkout. Relying only on the displayed page would therefore allow outdated information to enter the workflow. The backend validates the current item record and rejects an order containing an unavailable item.

Table Inventory

Each table has a branch association, table number, and seating capacity. The combination of branch and table number is unique, so numbering can be reused across different locations but not within the same location. The reservation engine uses this inventory to associate a booking with a specific physical table.

Table membership must be checked during booking. A valid table identifier is not sufficient if it belongs to a different branch from the one selected by the customer. Maintaining this relationship avoids inconsistent reservations and helps staff interpret the booking record correctly.

Operational Example

Consider two branches that both have a Table 1 and offer similarly named dishes. Their table records and menu items remain distinct because each resource belongs to a particular branch. A customer ordering from the first branch cannot combine its menu items with those of the second. A manager can adjust only the resources allowed by their assigned scope.

Deactivation Versus Deletion

Branch deletion includes cascading inventory cleanup. Deactivation is the documented choice when historical records must be retained while new business is blocked. The report does not assume that deactivation automatically cancels existing bookings or completes outstanding orders; those actions require separate operational handling.

7 Table Reservation Engine

Reservation Creation

A reservation associates a customer with a selected branch, table, start time, and duration. The backend verifies that the branch and table exist, that the table belongs to the branch, and that the requested start time is in the future. An inactive branch must not accept a new reservation. The service then evaluates existing bookings before creating a confirmed record and its initial status history.

Overlap Detection

Let s_n and e_n denote the requested start and end timestamps, and let d denote the duration in minutes. For timestamps represented in milliseconds, the requested end is

$$e_n = s_n + 60,000d.$$

An existing reservation with start s_e and end e_e conflicts with the new interval exactly when

$$s_e < e_n \quad \text{and} \quad e_e > s_n.$$

Both comparisons must hold. This identifies partial overlaps, contained intervals, and requests that span an existing booking. Cancelled reservations are excluded from the conflict query so that their time slots become available again.

End-Time Exclusivity

The strict inequalities permit one booking to begin at the exact time another ends. For example, a reservation from 19:00 to 20:30 does not conflict with a new booking starting at 20:30. This supports back-to-back use of a table without treating the shared boundary as an overlap.

Booking	Time Interval	Documented Outcome
First booking	19:00–20:30	Accepted when the table is otherwise free: HTTP 201.
Overlapping request	19:30–20:30	Rejected with HTTP 409 and a reservation-conflict error.
Following request	20:30–21:30	Accepted because the earlier end is exclusive.

Lifecycle and Ownership

A created reservation begins as **CONFIRMED**. The documented lifecycle permits completion or cancellation, with actions recorded in the history. Customers can access and manage only their own reservations, while authorized staff coordinate bookings within their permitted scope.

The interval rule is the core logical condition for conflict detection. The engine is designed to prevent double booking, but simultaneous-request behavior requires separate verification. The interval rule and example outcomes alone do not establish how database operations are coordinated under concurrent booking requests.

8 Cancellation and Rescheduling

Customer Cancellation Policy

The project applies an academic two-hour cutoff for customer reservation cancellations. The service first loads the reservation, checks ownership, and verifies that the record has not already been cancelled or completed. It then compares the reservation start time with the time of the cancellation request.

If t_r is the reservation start and t_c the current time, customer cancellation is allowed when

$$t_r - t_c \geq 2 \text{ hours.}$$

A request inside the two-hour window returns HTTP 409 with `CANCELLATION_WINDOW_EXPIRED`. Exactly two hours before the start satisfies the documented condition. Authorized managers and administrators have an override for operational situations, subject to their access scope.

Illustrative Boundary Cases

For a booking beginning at 19:00, a customer request at 16:30 is within the allowed cancellation period. A request at 17:00 meets the cutoff exactly, while a request at 17:01 is too late under this policy. These examples explain the comparison rather than represent newly executed tests.

On successful cancellation, the reservation becomes `CANCELLED` and the action is appended to its history. Because cancelled records are excluded from future overlap searches, the slot is released without deleting the original booking. This retains a record of the customer's interaction while allowing another customer to reserve the time.

Rescheduling Procedure

1. Load the reservation and verify the caller's ownership or authorized staff access.
2. Reject changes to a cancelled or completed reservation.
3. Validate the replacement start time and duration, including the future-time requirement.
4. Search for conflicts on the replacement slot, excluding the reservation being changed.
5. If the slot is free, update the start, duration, and end time, then append the change to the history.

Preserving the Original Booking

The important design choice is to check the proposed slot before modifying the current reservation. If the new time overlaps another booking, the request returns a conflict and the original booking remains unchanged. The customer therefore does not lose a valid reservation merely because an attempted replacement is unavailable.

This workflow is described as atomic rescheduling. Its intended behavior combines pre-validation with preservation of the original record on a conflict; transaction or locking details still require implementation-level verification. Cancellation and rescheduling also remain distinct operations: the stated two-hour rule specifically governs customer cancellation, so this report does not extend it to rescheduling without supporting documentation.

9 Food Ordering and Price Preservation

Supported Order Types

The system accepts `DINE_IN` and `TAKEAWAY` orders. Dine-in orders connect the food request to the restaurant’s table or reservation context. Takeaway orders follow the same menu validation and preparation workflow without the dine-in service charge. Both types belong to a selected branch and retain the customer relationship needed for access control and history.

Order Validation Sequence

Before accepting an order, the backend checks the branch and every requested menu item. Each item must exist, belong to the selected branch, and be available. Quantities must be integers between 1 and 50. If any requested item is unavailable, the entire request is rejected with HTTP 400 and `ITEM.UNAVAILABLE`, rather than silently creating a partial order.

This all-or-nothing validation makes the accepted order match what the customer requested. Cross-branch validation is equally important: two items with valid identifiers must still be rejected when they belong to different locations. An inactive branch also refuses new orders, consistently with its reservation restriction.

Authoritative Price Calculation

The server retrieves item prices from the database rather than accepting the browser’s totals as authoritative. For each item i , with unit price p_i and quantity q_i , the calculations begin with

$$\text{lineTotal}_i = p_i q_i, \quad \text{subtotal} = \sum_i p_i q_i.$$

Client-supplied subtotal or total values are discarded. The dashboard may display a cart estimate, but the server determines the accepted bill. This protects the recorded transaction against modified requests and stale displayed prices.

Embedded Purchase Snapshots

The accepted order embeds each item’s identifier, name, unit price, quantity, and line total. These snapshots describe the purchase at the time of placement. For example, if a dish costs 200.00 when ordered and is later changed to 220.00 in the menu, the earlier order retains 200.00 as its unit price.

This separates the current catalog from historical transactions. Staff can update the menu without rewriting previous purchase terms, while customers and managers can still review the original itemized record. The order also carries a status history that records how it progresses through preparation and fulfillment.

Order Access and Follow-Up

Customers can retrieve their own orders and related summaries. Authorized staff use the same order records for preparation and service. The shared record links placement, workflow, billing, and later feedback instead of treating each stage as an unrelated activity. Detailed bills and operational summaries are exposed through dedicated order endpoints with their respective access checks.

10 Kitchen Queue and Order Workflow

Kitchen Display Queue

The kitchen endpoint, `/api/kitchen/orders`, returns active orders in `PLACED` or `PREPARING` status and sorts them by creation time in ascending order. This FIFO presentation places the oldest ticket first and provides a clear preparation sequence. Branch filtering keeps the display relevant to a particular kitchen.

Tickets contain the information needed to prepare food: order type, table number where applicable, item names, and quantities. Customer identifiers, names, and email addresses are removed. The queue is therefore an operational view rather than a copy of the full customer or billing record. Once an order becomes ready, it no longer meets the active queue’s stated filter.

Permitted State Transitions

PLACED	→	PREPARING
PREPARING	→	READY
READY	→	SERVED (dine-in)
READY	→	DELIVERED (takeaway)
PLACED	→	CANCELLED

The state machine prevents arbitrary jumps, such as changing a newly placed order directly to served. Cancellation is permitted from the placed state before cooking begins. Served, delivered, and cancelled are terminal states, so they cannot be advanced or reopened through the documented workflow. Invalid transitions return HTTP 409 with `INVALID_STATUS_TRANSITION`.

Role-Specific Actions

Kitchen staff can move placed orders to preparing and preparing orders to ready. They cannot mark orders as served or delivered. Managers and administrators can perform the permitted service and fulfillment transitions within their authorized scope. Customers are prohibited from making staff preparation-status updates.

The customer dashboard is described as offering order cancellation while an order is placed, whereas the status-update endpoint is restricted to staff roles. A separate customer order-cancellation route has not been verified for this report. That interface detail should therefore be confirmed in the application rather than treating general customer status updates as authorized.

Audit History and Coordination

Each status change is recorded with a timestamp, acting user identifier, and optional remarks in `Order.statusHistory`. This allows an operational summary to show not only the latest state but also the sequence of actions leading to it. The queue and state machine together improve coordination: the queue determines which work is active, while transition rules determine which actions are valid next.

11 Billing and Order Summaries

Centralized Billing Rules

The billing engine calculates item totals, tax, service charge, and the final amount on the server. The configured defaults are a tax rate of 5% and a dine-in service charge of 5%, configured through `TAX_RATE` and `SERVICE_CHARGE_RATE`. Takeaway orders have a zero service charge. These percentages are the project’s configured academic rules, not a statement about statutory restaurant charges.

Let S be the subtotal, r_t the tax percentage, and r_s the applicable service-charge percentage. The documented calculations are

$$\begin{aligned} S &= \sum_i \text{unitPrice}_i \times \text{quantity}_i, \\ T &= S \times r_t / 100, \\ C &= S \times r_s / 100, \\ \text{Total} &= S + T + C. \end{aligned}$$

Both percentages apply to the subtotal. For takeaway, $r_s = 0$. Monetary calculations are rounded to two decimal places using a rounding helper.

Illustrative Bill

Assume an order contains two meals at 200.00 each and two beverages at 50.00 each. Its subtotal is $2(200.00) + 2(50.00) = 500.00$. The comparison below uses the documented default rates and illustrative monetary units.

Component	Dine-In	Takeaway
Meals	400.00	400.00
Beverages	100.00	100.00
Subtotal	500.00	500.00
Tax at 5%	25.00	25.00
Service charge	25.00	0.00
Final total	550.00	525.00

Bill and Summary Endpoints

The route `/api/orders/:id/bill` returns an itemized financial view, while `/api/orders/:id/summary` provides the operational summary, including status history. These views use the stored order information instead of reconstructing old item prices from the current menu. This keeps past purchases understandable after catalog changes.

Privacy and Interpretation

Billing access is permitted for the owning customer and authorized managers or administrators, not kitchen staff or unrelated customers. Server-calculated totals also remain authoritative even if the frontend displays an incorrect or manipulated estimate. The resulting bill records an amount due under the configured rules; online payment collection, settlement confirmation, and refunds are outside the established scope, so generating a bill should not be interpreted as processing a payment.

12 Customer History and Verified Feedback

Paginated Histories

The customer history module provides separate views for reservations and orders. Order history supports status, order type, branch, and date-range filters. Reservation history supports status, branch, and date-range filters. Pagination uses `page`, `limit`, `total`, and `totalPages`, allowing the dashboard to present manageable result sets rather than loading every record at once.

Logged-in customers can use `/api/customers/orders` and `/api/customers/reservations`. Identifier-based variants also enforce ownership. A date filter whose start is after its end is rejected as an invalid date range. Filtering and pagination therefore operate within the user's permitted records rather than bypassing access control.

Feedback Eligibility

Reviews are tied to completed orders. A dine-in order must be **SERVED**, while a takeaway order must be **DELIVERED**. The customer submitting the review must own the order. Attempting to review an unfinished order returns HTTP 409 with an `order-not-eligible-for-feedback` code. This links feedback to an actual completed interaction rather than an unverified visit claim.

Rule	Requirement
Rating	An integer from 1 to 5; values outside this range are invalid.
Comment	Optional text with a documented maximum of 500 characters.
Ownership	The submitting customer must own the completed order.
Uniqueness	A unique order–customer index prevents duplicate feedback for the same purchase.
Management	Customers may update their own feedback; deletion follows customer-ownership or administrator permissions.

Review Workflow

A customer first opens a completed order from history and submits a rating with an optional comment through `POST /api/feedback`. The backend checks the order, ownership, completion state, rating range, and duplicate constraint before storing the review. A repeated submission is rejected with `FEEDBACK_ALREADY_EXISTS`. Feedback can also be retrieved in the context of its order.

Value to Customers and Managers

History makes earlier bookings and orders easier to revisit, while feedback provides a structured way to comment on a completed experience. For managers, branch-linked reviews contribute to average-rating and rating-distribution reports. Verified eligibility improves the connection between a review and a transaction, although it does not by itself prove that every review is objective or representative of all customers. Ratings should therefore be interpreted alongside review counts and operational information.

13 Manager Analytics and Frontend Dashboard

Branch-Scoped Reports

The reporting module uses MongoDB aggregation pipelines to summarize operational records. Managers receive results within their assigned branches, while administrators have global visibility. Applying access scope to reporting is important because aggregate figures can expose branch information even when individual order records are protected.

The report endpoints share the prefix `/api/manager/reports`. Their documented purposes are summarized below.

Endpoint Suffix	Report Purpose
<code>/summary</code>	Executive overview of key operational indicators.
<code>/sales</code>	Sales, completed orders, and realized-revenue summaries.
<code>/popular-dishes</code>	Item ranking using quantities sold and revenue information.
<code>/peak-hours</code>	Distribution of order activity by hour.
<code>/reservation-peak-hours</code>	Distribution of reservation demand by hour.
<code>/ratings</code>	Average rating and distribution across one to five stars.

Aggregation Approach

The aggregation design uses `$unwind` to examine embedded order items when ranking dishes and `$hour` to group demand by hour. These operations turn transaction records into summaries that can support menu planning and staffing discussions. Sales and completed-order measures help describe activity, while rating distributions show information that an average alone could conceal.

The reports are described as real-time because they present operational data rather than fixed figures. A refresh interval, transport mechanism, and measured reporting latency have not been established for this report. No live revenue or customer-satisfaction figures are invented here.

Role-Oriented Browser Interface

Express serves a single-page dashboard built with HTML, CSS, and vanilla JavaScript. Its authentication portal includes registration and demonstration logins for the four roles. The customer portal presents branch selection, menu categories, availability indicators, a food cart, table reservation controls, histories, and feedback actions.

The kitchen interface displays active tickets and preparation controls with a branch filter. The manager dashboard presents metric cards, top-selling dishes, peak ordering information, and branch active/inactive controls. These screens bring the API workflows together in a form suitable for a live academic demonstration.

Interpreting Operational Information

A popular-dishes report could help staff identify frequently ordered items, while peak-hour summaries could inform staffing discussions. Such uses are potential management benefits, not independently measured project outcomes. The quality of any decision still depends on correct underlying records, appropriate report filters, and clear definitions of the figures shown.

14 Setup, Demonstration and Testing

Documented Setup Procedure

The original application requires Node.js, npm, and a MongoDB connection. Setup involves installing dependencies, copying the environment template, seeding demonstration records, and starting the server. The following commands belong to that application repository, not to this LaTeX report workspace.

```
npm install
cp .env.example .env
npm run seed
npm run dev
```

The alternative start command is `npm start`, and the documented test command is `npm test`. Configuration includes `PORT` (default 5001), `NODE_ENV`, `MONGODB_URI`, `JWT_SECRET`, and `JWT_EXPIRES_IN`. The seed step populates users for each role, branches, menus, tables, and sample transactions. Demonstration credentials should remain restricted to demonstration use.

Suggested Demonstration Sequence

A concise presentation can begin with customer login and branch selection, followed by a successful reservation and an overlapping request. The customer then places an order, kitchen staff advance it through preparation, and a manager records completion. Finally, the customer views the bill and submits feedback, while the manager opens the resulting history and analytics views. These steps connect individual modules into one understandable user journey.

Reported Verification Coverage

The reported integration-test result is 149 out of 149 assertions passing with zero failures. The project acceptance matrix covers 13 modules. This is a reported suite-wide count, not 149 separate tests for each module. Representative documented rules for verification include:

Scenario	Expected Behavior
Overlapping booking	Reject with HTTP 409; allow an exact end-to-start boundary.
Unavailable order item	Reject the whole request with <code>ITEM.UNAVAILABLE</code> .
Invalid workflow change	Reject skipped stages or changes from a terminal state.
Late customer cancellation	Enforce the two-hour cutoff; preserve valid staff override.
Duplicate or early review	Reject duplicate feedback and feedback before completion.
Out-of-scope access	Deny another customer's records or an unassigned branch.

Verification Limit

The project includes a Postman collection and supporting testing documentation for reproducing API checks. The application suite was not run, and no new screenshots, performance measurements, or runtime results are claimed. The stated pass count should be confirmed against the original repository when preparing the final demonstration or submission evidence.

15 Outcomes, Limitations and Conclusion

Documented Outcomes

The reported project completion covers authentication, menu management, table inventory, reservations, ordering, kitchen operations, workflow control, billing, cancellation, customer history, feedback, branch controls, and analytics. Its principal contribution is integrating these activities around shared restaurant records and consistent backend rules.

The design addresses several of the original problems directly. Interval checks handle overlapping booking requests; server-side item validation keeps orders tied to available dishes at the correct branch; embedded prices preserve historical purchase terms; and the order state machine coordinates preparation and fulfillment. Ownership and branch scoping separate customer and staff responsibilities, while retained histories support later review.

Limitations and Documentation Gaps

This report is based on documentation rather than a source-code audit or an executed application. The reported test count does not establish production reliability, concurrent booking safety, or performance at large scale. Details such as the live dashboard refresh mechanism and deployment monitoring remain unverified. Node.js version requirements and customer order-cancellation behavior should also be confirmed against the actual implementation.

The configured tax and service-charge percentages are academic application rules. Online payment settlement, external delivery tracking, automated notifications, and production recovery procedures are not established as implemented features within the established project scope. Existing billing and takeaway support should not be confused with these wider integrations.

Possible Future Enhancements

Useful extensions could include booking reminders, wait-list handling for cancelled slots, and optional online payment integration. A stronger deployment evaluation could add concurrency tests for reservations, load tests for kitchen and reporting queries, and checks for backup restoration. Dashboard refresh behavior and accessibility could also be documented and evaluated more explicitly. These are proposed improvements, not claims about the current project.

Conclusion

The Restaurant Table Reservation & Food Ordering System demonstrates how a JavaScript backend can organize a multi-role restaurant workflow. It combines REST APIs, validated data models, role-aware access, interval-based booking rules, controlled order progression, and aggregation reports in one application. The browser dashboard makes these backend capabilities accessible to customers and operational staff.

Within the documented academic scope, the project provides a coherent foundation for managing reservations and food orders across branches. Its strongest design feature is the use of shared records and explicit business rules from booking through feedback. Further evaluation should focus on confirming the documented behavior in the original application and assessing the additional safeguards needed for real-world deployment.